# Aula 17: Arquitetura de Computadores – Unidades de processamento gráfico de uso geral

Prof. Hugo Puertas de Araújo
hugo.puertas@ufabc.edu.br
Sala: 509-2 (5º andar / Torre 2)

# Arquitetura de Computadores

# Objetivos de aprendizagem

- Apresentar uma visão geral sobre a CUDA.
- Entender a diferença entre uma GPU e uma CPU.
- Descrever os elementos básicos de uma arquitetura GPU típica.
- Discutir sobre quando usar uma GPU como um coprocessador.

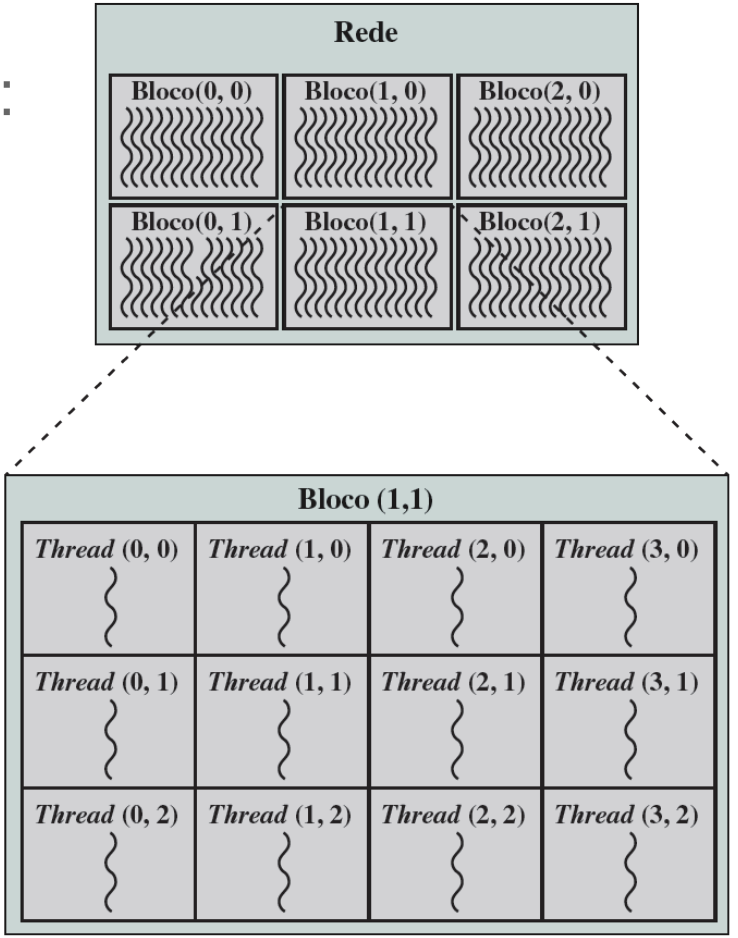
# Noções básicas sobre a CUDA

- A CUDA (do inglês, **C**ompute **U**nified **D**evice **A**rchitecture) é uma plataforma de computação paralela e de modelo de programação criada pela NVIDIA e implementada pelas unidades de processamento gráfico (GPUs) que elas produzem.
- A CUDA C é uma linguagem baseada em C/C++.
- Um programa em CUDA pode ser dividido em:
  i. o código a ser executado pelo host (CPU);
  ii. o código a ser executado pelo dispositivo (GPU); e
  iii. o código relacionado à transferência de dados entre o host e o dispositivo.


CMCC Centro de Matemática, Computação e Cognição

# Noções básicas sobre a CUDA

- Relação entre threads, blocos e uma rede:




**Thread → uma instância de código paralelo**
**Bloco → conjunto de threads atribuído a um processador**
**Rede → a própria GPU**

# Noções básicas sobre a CUDA

- Termos da CUDA para mapeamento de equivalência de componentes de hardware de GPU:

| Termo de CUDA | Definição | Componente equivalente de hardware de GPU |
|---|---|---|
| Kernel | Código paralelo na forma de uma função a ser executada na GPU | Não se aplica |
| *Thread* | Uma instância do kernel na GPU | *Core* processador de uma GPU/CUDA |
| Bloco | Um grupo de *threads* atribuído a um MS em particular | Multiprocessador de CUDA (MS) |
| Rede | A GPU | GPU |

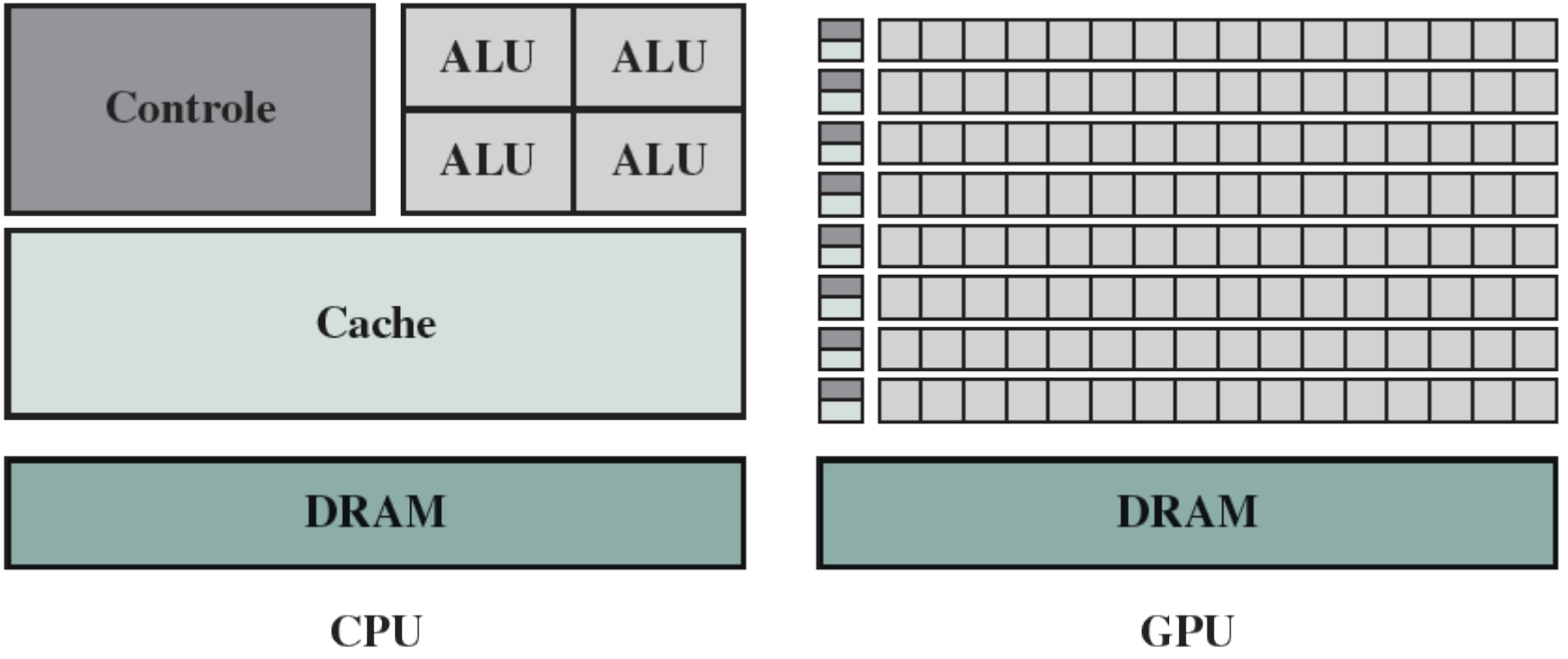
**multiprocessadores de streaming (MS)**



CMCC Centro de Matemática, Computação e Cognição

# GPU versus CPU

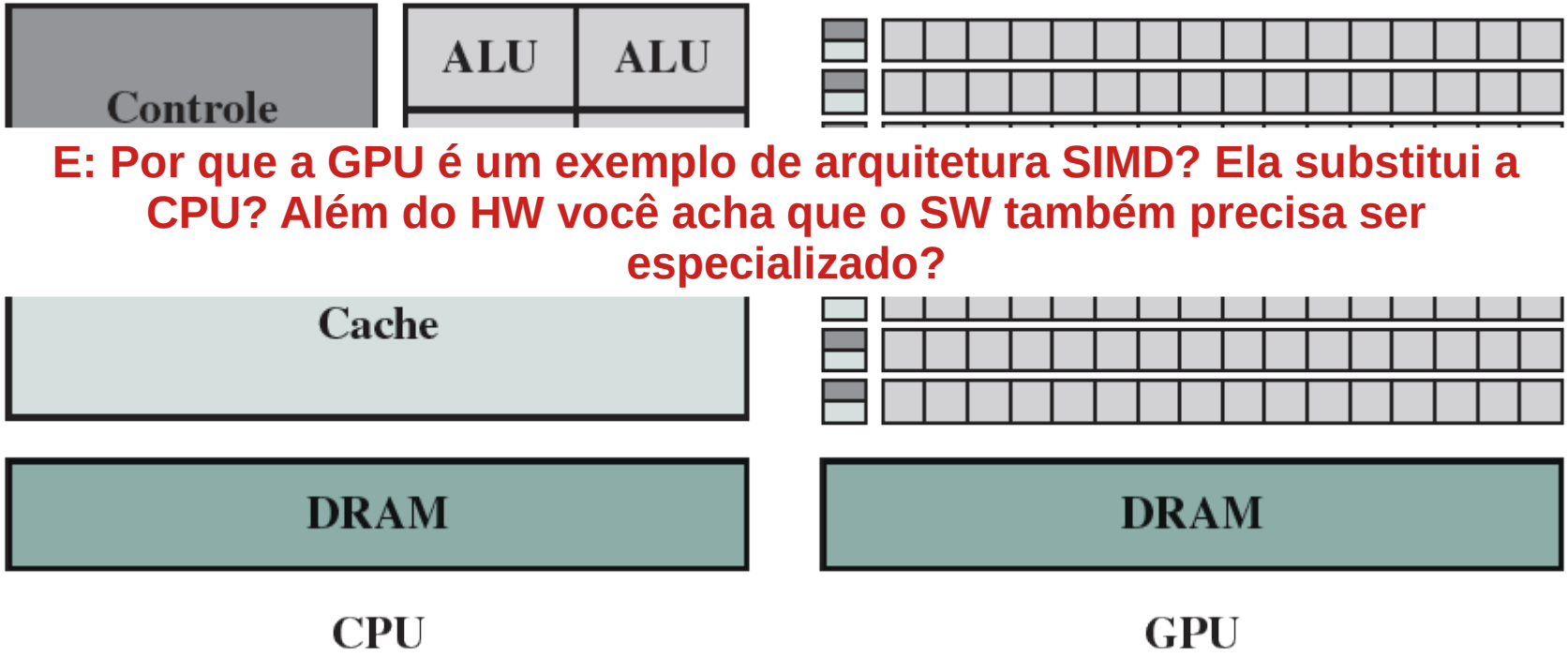
- Dedicação de transístores/área de silício de uma CPU versus uma GPU:



# GPU versus CPU

Exercício

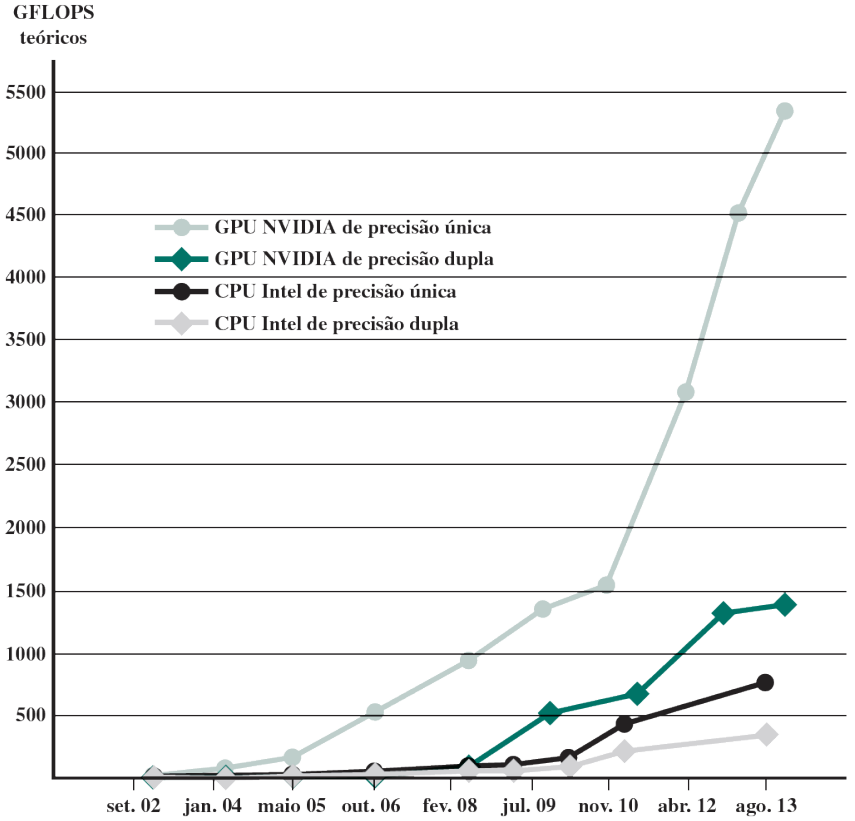
- Dedicação de transístores/área de silício de uma CPU versus uma GPU:




**E: Por que a GPU é um exemplo de arquitetura SIMD? Ela substitui a CPU? Além do HW você acha que o SW também precisa ser especializado?**

# GPU versus CPU

- Operações de ponto flutuante por segundo para CPU e GPU:



# Visão geral da arquitetura de uma GPU

- A evolução histórica da arquitetura de uma GPU pode ser dividida em três fases ou eras principais:
  i. A primeira fase cobriria os anos iniciais da arquitetura da GPU (do início da década de 1980 ao final da década de 1990) – **processamento não programável e fixo**.
  ii. A segunda fase abrangeria a modificação iterativa da arquitetura resultante da GPU, fase 1, de um pipeline de hardware especializado e fixo para um **processador totalmente programável (CUDA – 2006)**.
  iii. A terceira fase abrange como a arquitetura GPU/GPGPU torna um **coprocessador SIMD** altamente paralelizado e acessível.


CMCC Centro de Matemática, Computação e Cognição

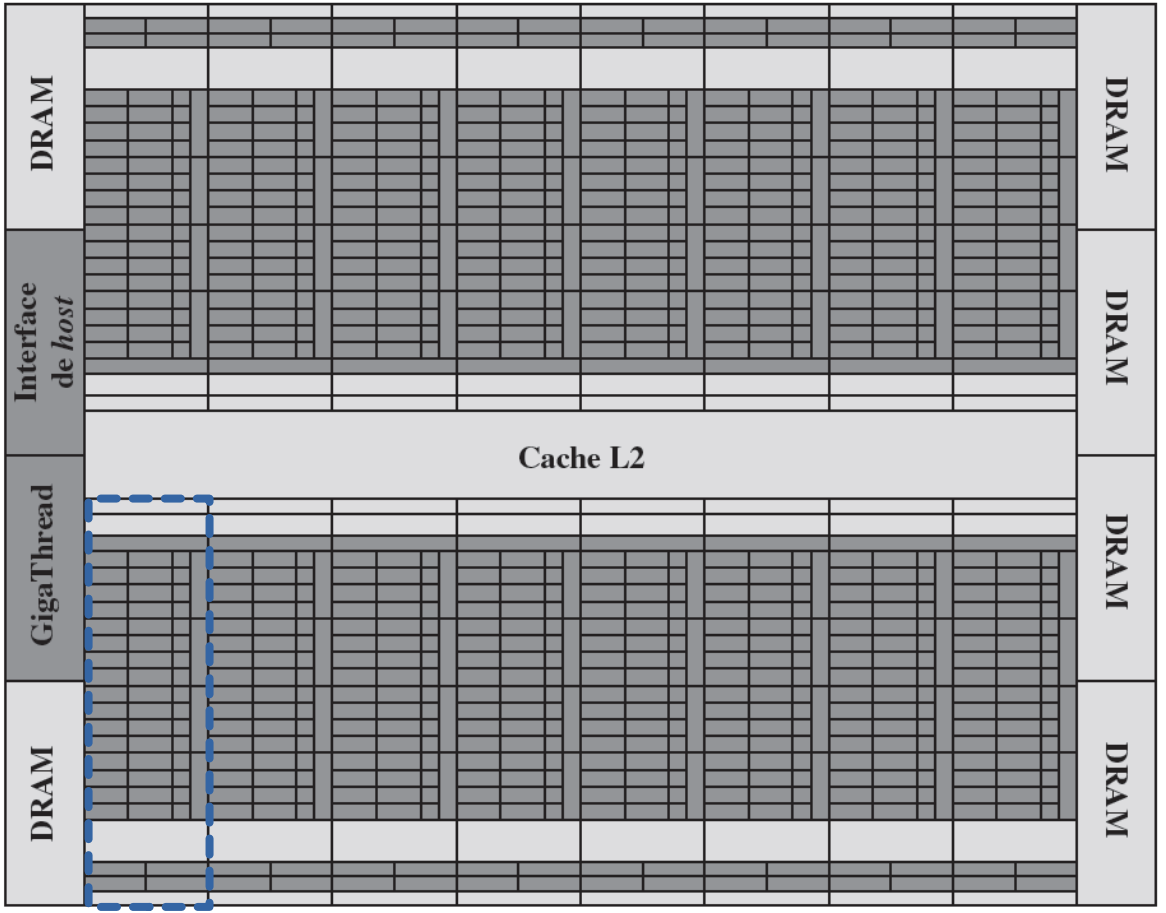
# Visão geral da arquitetura de uma GPU

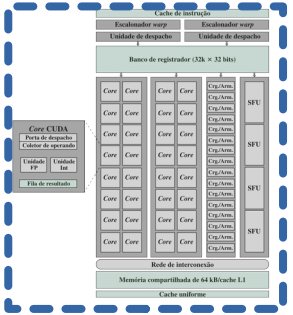
- Arquitetura Fermi da NVIDIA:

**I/F p/ DRAM externa**

**I/F PCIe GPU ↔ CPU**

**Escalonador**



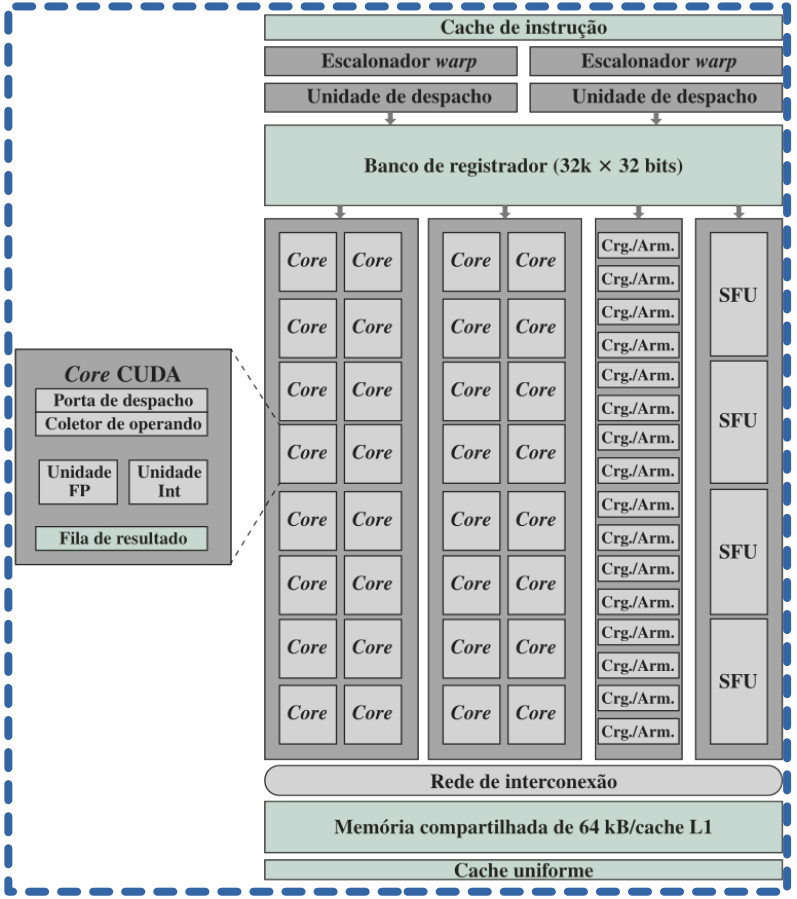



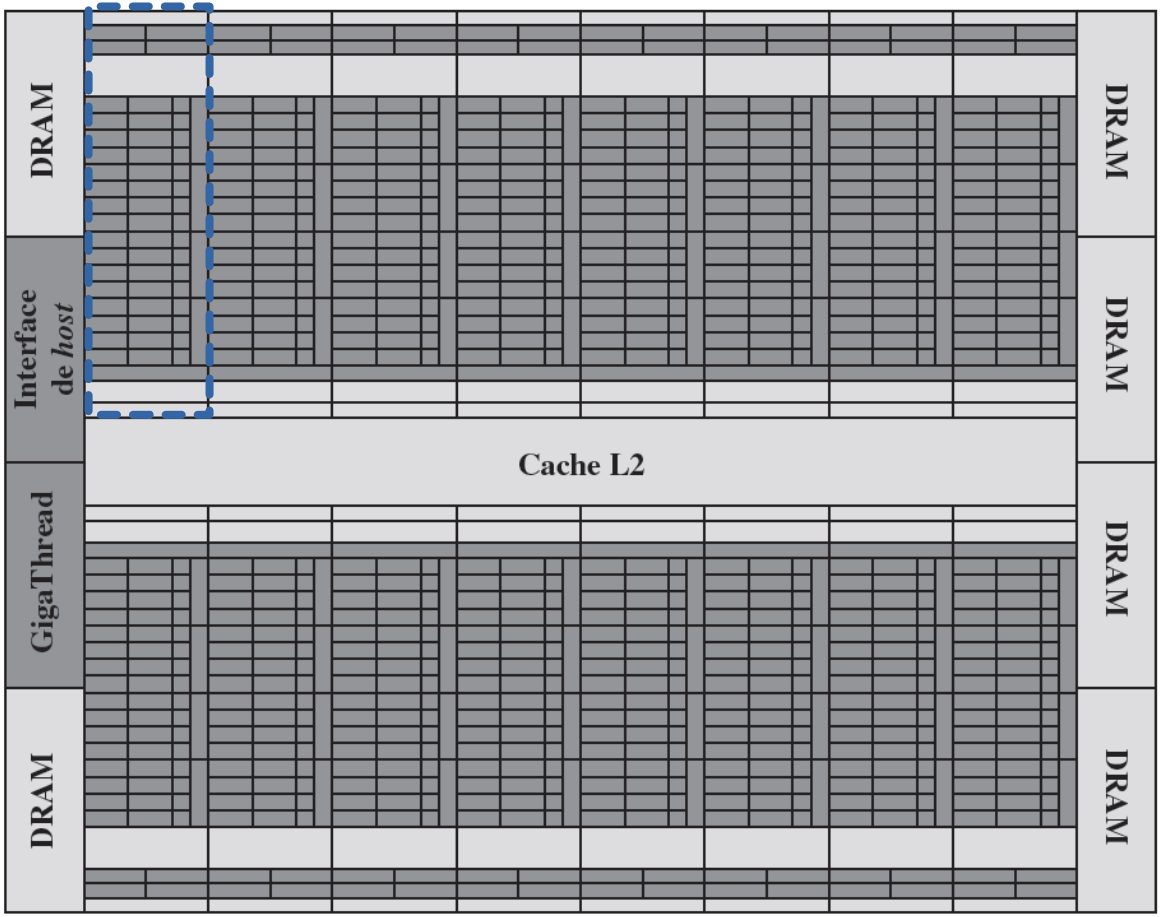

**16 MS's, em que cada MS contém um grupo de 32 cores CUDA**



CMCC
Centro de Matemática, Computação e Cognição

# Visão geral da arquitetura de uma GPU



# Detalhes da arquitetura do multiprocessador de streaming

- Os componentes básicos para um único MS são:
  - Cores de processadores de GPU (total de 32 cores CUDA).
  - Escalonador warp e porta de despacho.
  - Dezesseis unidades de carga/armazenamento.
  - Quatro SFUs.
  - Registradores 32k × 32 bits.
  - Memória compartilhada e cache L1 (64 kB no total).

**(arquitetura Fermi da NVIDIA)**

# Detalhes da arquitetura do multiprocessador de streaming

- **ESCALONADOR WARP(*) DUPLO**. O escalonador de warp duplo vai então separar cada bloco de thread que está processando em warps, que são agrupamentos de 32 threads que começam no mesmo endereço inicial e seus IDs de thread são consecutivos.
- **CORES CUDA**. Existe um total de 32 cores CUDA dedicados a cada MS na arquitetura Fermi. Cada core CUDA tem dois pipelines ou caminhos de dados separados: uma unidade pipeline de inteiro (INT) e uma unidade de pipeline de ponto flutuante (PF).

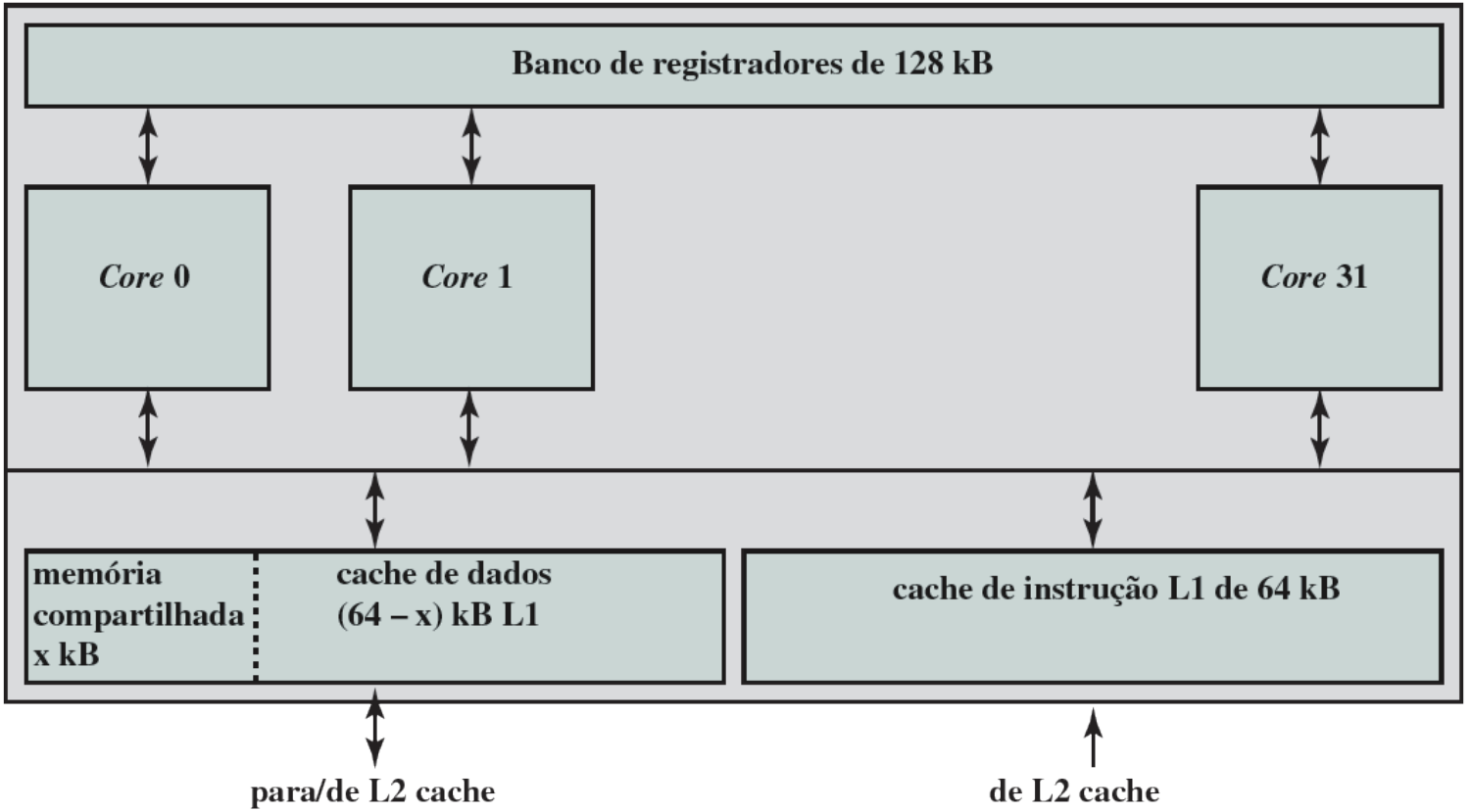
**(*) warp é uma unidade básica de execução paralela dentro de um SM**

# Detalhes da arquitetura do multiprocessador de streaming

- **UNIDADES DE FUNÇÃO ESPECIAIS**. Cada MS tem quatro SFUs. A SFU apresenta operações transcendentais, como cosseno, seno, recíproca e raiz quadrada, em apenas um ciclo de clock.
- **UNIDADES DE CARGA E ARMAZENAMENTO**. Cada uma das 16 unidades de carga e armazenamento do MS calcula a fonte e o endereço de destino para um único thread por ciclo de clock.
- **REGISTRADORES, MEMÓRIA COMPARTILHADA E CACHE L1.** Cada MS tem seu próprio conjunto (on-chip) dedicado de registradores e de memória compartilhada/bloco de cache L1.

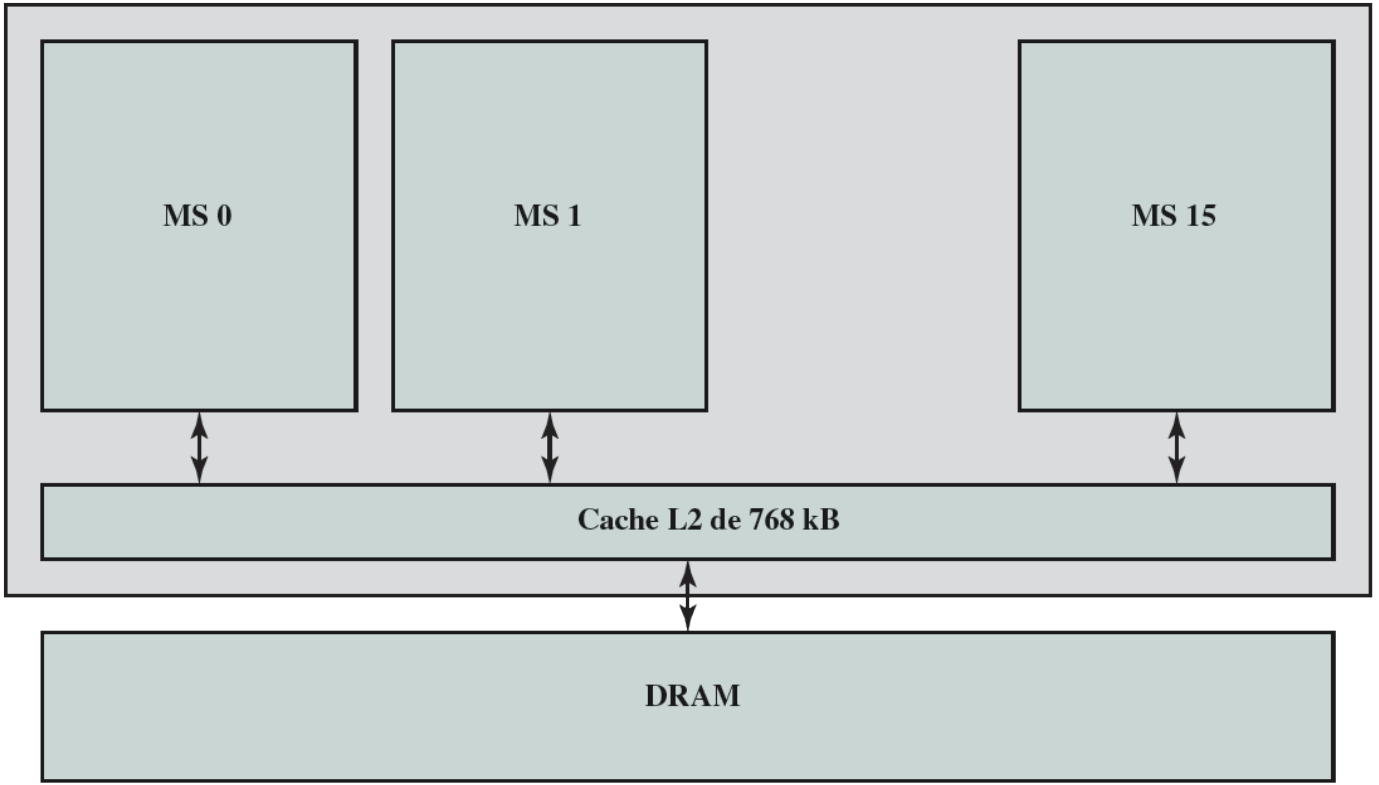
# Detalhes da arquitetura do multiprocessador de streaming

- Arquitetura da memória do MS:



# Detalhes da arquitetura do multiprocessador de streaming

- Arquitetura completa da memória:



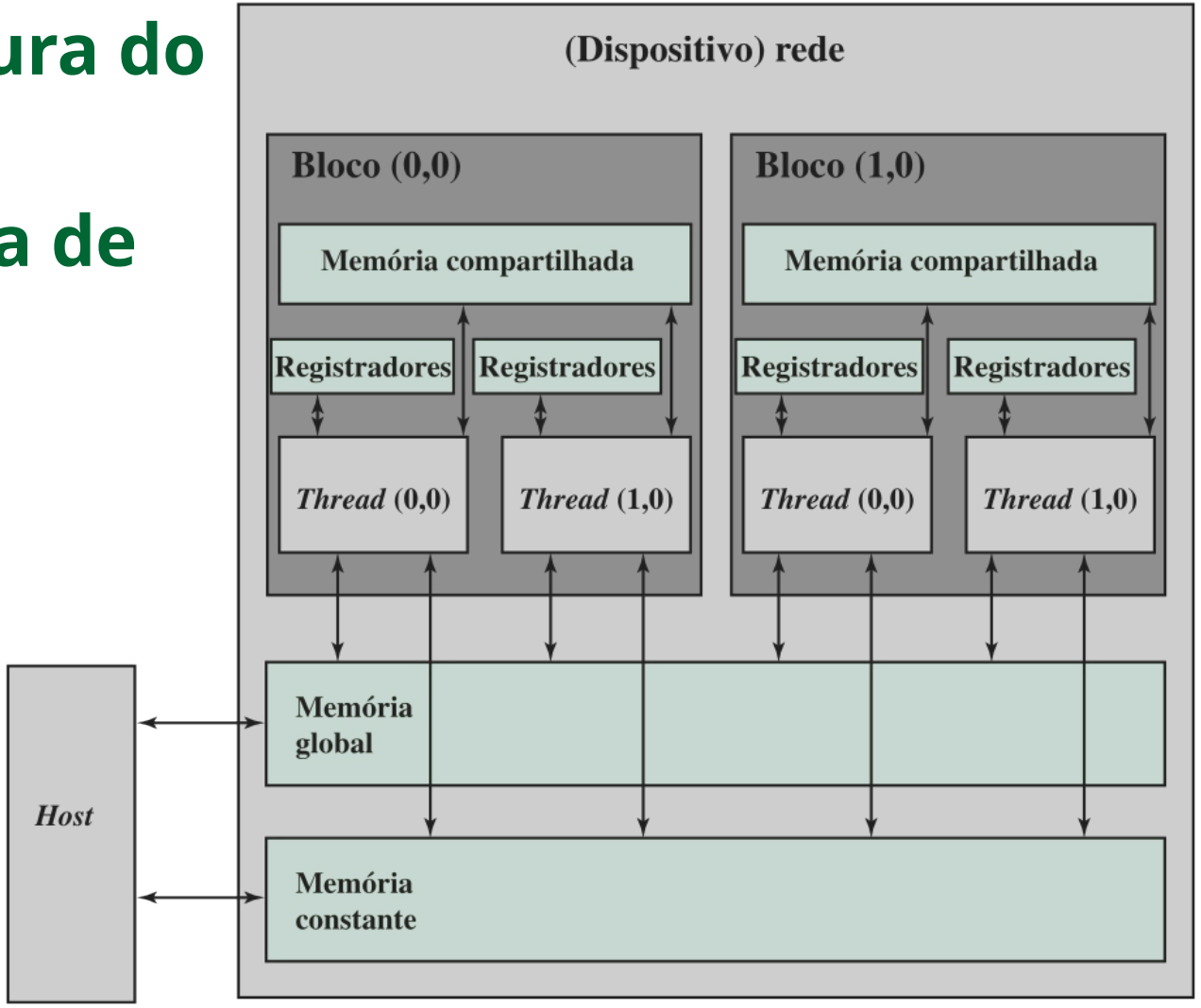
# Detalhes da arquitetura do multiprocessador de streaming: Hierarquia de memória

| Tipo de memória | Tempos de acesso relativo | Tipo de acesso | Escopo | Vida útil dos dados |
|---|---|---|---|---|
| Registradores | O mais rápido. On-chip | L/E | *Thread* única | *Thread* |
| Compartilhada | Rápido. On-chip | L/E | Todas as *threads* em um bloco | Bloco |
| Local | Entre 100 e 150 vezes mais lento que a compartilhada e o registrador. Off-chip | L/E | *Thread* única | *Thread* |
| Global | Entre 100 e 150 vezes mais lento que a compartilhada e o registrador. Off-chip. | L/E | Todas as *threads* e *host* | Aplicação |
| Constante | Entre 100 e 150 vezes mais lento que a compartilhada e o registrador. Off-chip | L | Todas as *threads* e *host* | Aplicação |
| Textura | Entre 100 e 150 vezes mais lento que a compartilhada e o registrador. Off-chip | L | Todas as *threads* e *host* | Aplicação |

# Detalhes da arquitetura do multiprocessador de streaming: Hierarquia de memória

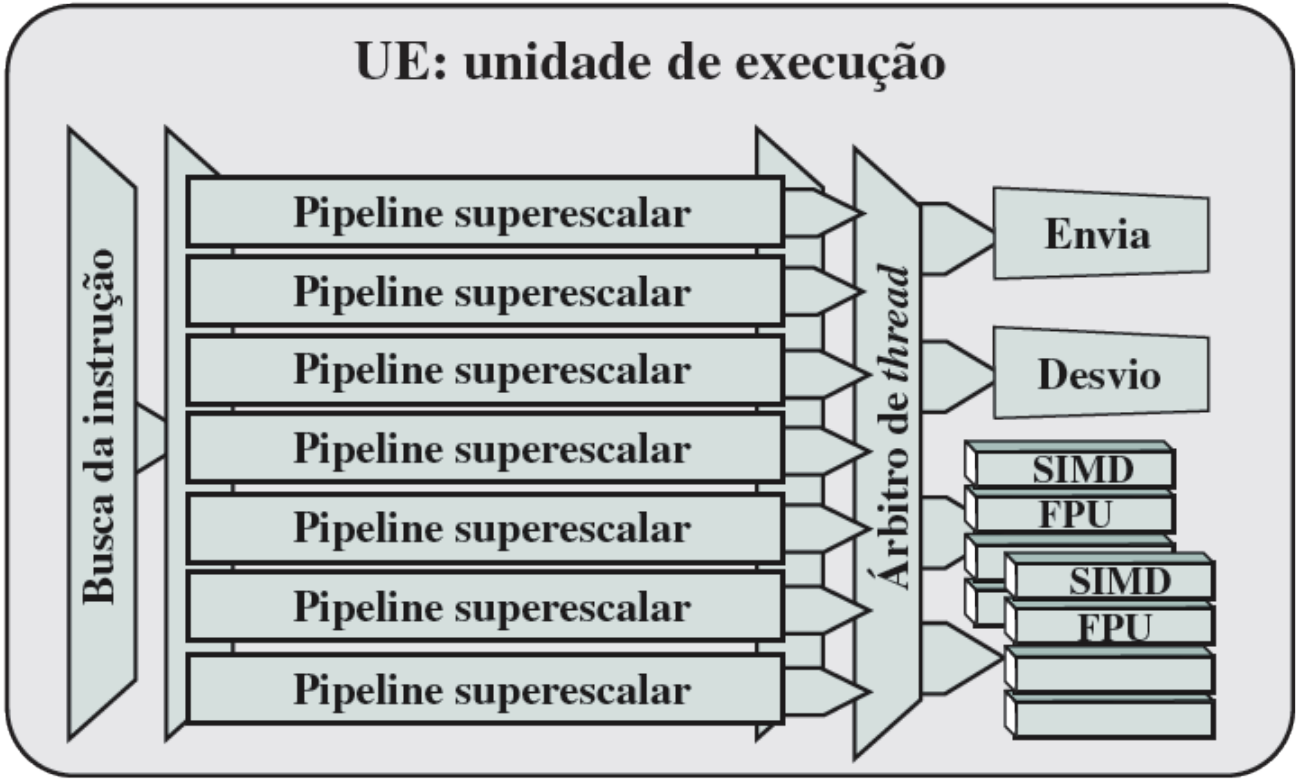
O único modo em que threads dentro de um MS em particular podem cooperar entre si (via compartilhamento de dados) é por meio de memória compartilhada. Isso geralmente é realizado pelo <u>programador.</u>

(O gerenciamento da memória é atributo do programador, o contrário da programação em CPU)



# GPU Gen8 da Intel

- Unidade de execução Gen8 da Intel:



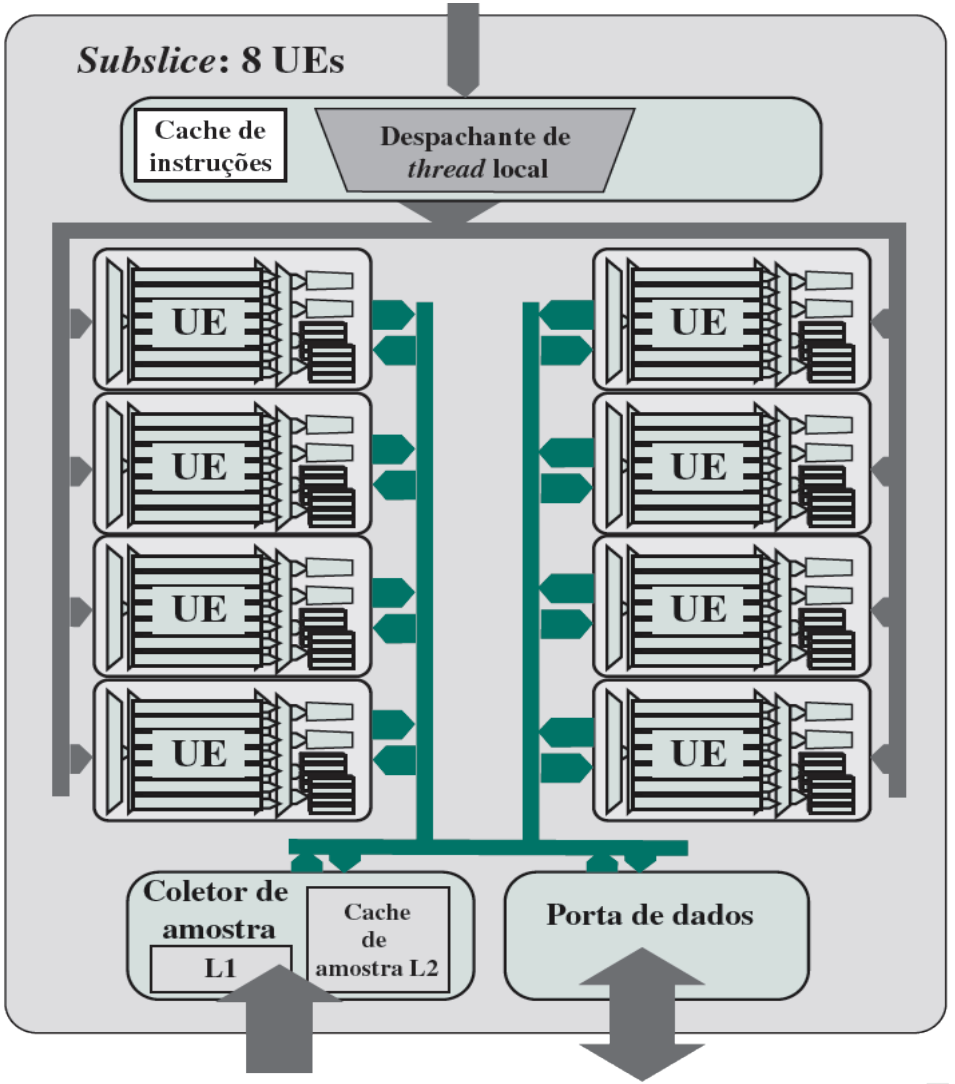
# GPU Gen8 da Intel

- As UEs são organizadas em subslices, que podem conter até oito UEs.
- Cada subslice contém sua própria unidade de despacho de thread local e seu próprio suporte a caches de instruções.
- De tal modo, um único subslice tem recursos de hardware dedicados e bancos de registradores para um total de 56 threads simultâneos.
- Um subslice também inclui uma unidade chamada de coletor de amostra, com suas próprias caches L1 e L2.


CMCC
Centro de Matemática, Computação e Cognição

# GPU Gen8 da Intel

- Subslice do Gen8 da Intel:



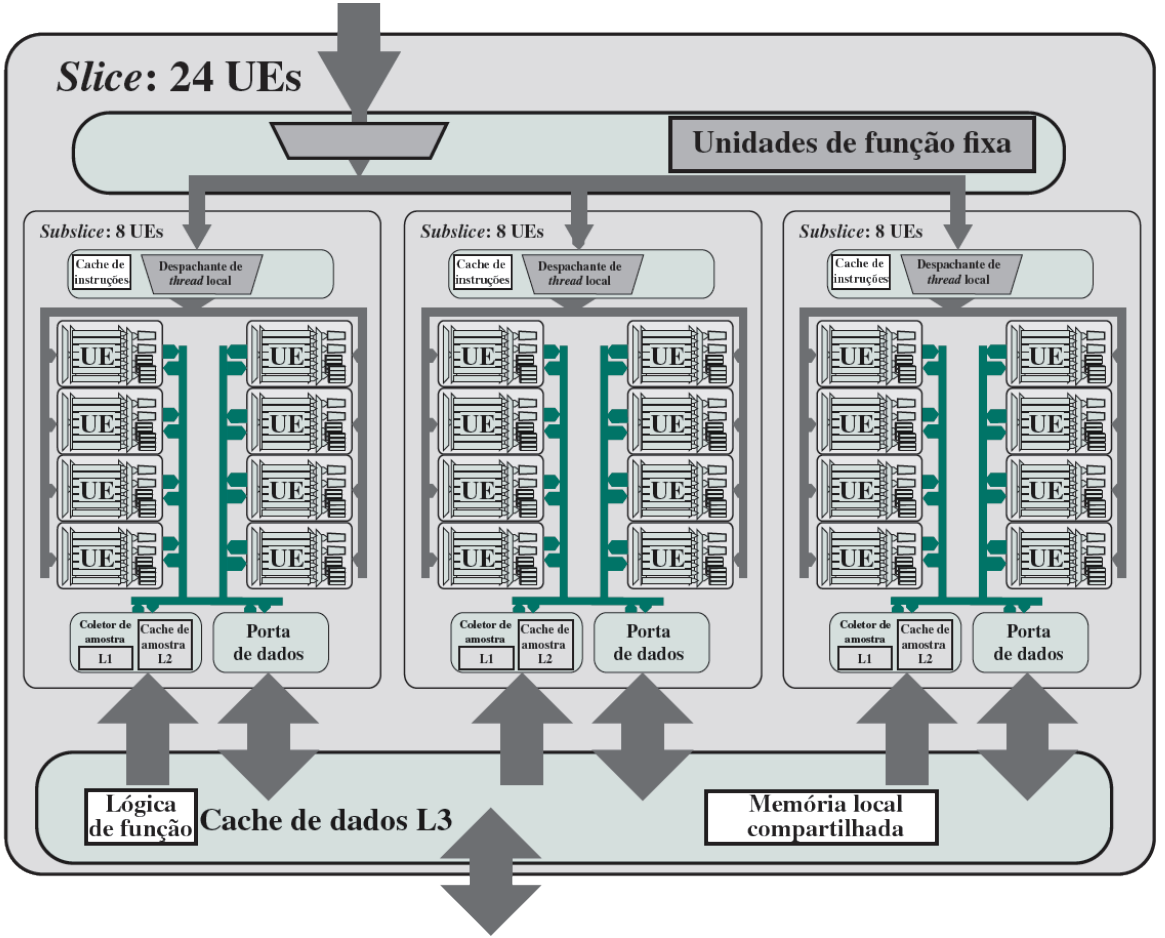
# GPU Gen8 da Intel

- Para criar variantes de produto, os subslices podem ser dispostos em clusters chamados de slice.
- Atualmente, até três subslices podem ser organizados dentro de um único slice para um total de 24 UEs.
- Além dos subslices, o slice inclui lógica para o roteamento de despacho de thread, outra função lógica para otimizar o processamento de dados gráficos, uma cache L3 compartilhada e uma estrutura local menor compartilhada.
- A última é visível (memória endereçável) às UEs e é útil para compartilhar variáveis temporárias.


CMCC
Centro de Matemática, Computação e Cognição

# GPU Gen8 da Intel

- Slice do Gen8 da Intel:



# Quando usar uma GPU como um coprocessador

- Programas que têm uma ou mais partes altamente paralelizáveis de código, que podem ser replicadas em milhares de threads compactos para funcionar com conjuntos grandes de dados em concorrência, são os melhores candidatos a acelerar seu tempo de execução em um sistema de GPGPU
- Em geral, o código original serial está na forma de uma grande iteração for, ou alguns loops for embarcados, que apresentam cálculos em equações que não têm dependência de dados entre as iterações (por exemplo, aritmética de matriz)
- Uma vez que um segmento de código massivamente paralelizável seja identificado, ele precisa então ser convertido a partir do código serial em código paralelo ou em um kernel CUDA


CMCC
Centro de Matemática, Computação e Cognição